

RankGraph

Lifecycle Co-Design for Billion-Node Graph Learning in Recommendation



Graph construction, training and serving are coupled — prior work optimizes one stage at a time. We co-design all three.

3.8×

Recall@5 vs. GAT + Deep Graph Infomax (user retrieval)

2.1×

Recall@100 vs. PyTorch-BigGraph (item retrieval)

-83%

U2U serving infrastructure cost — no online KNN

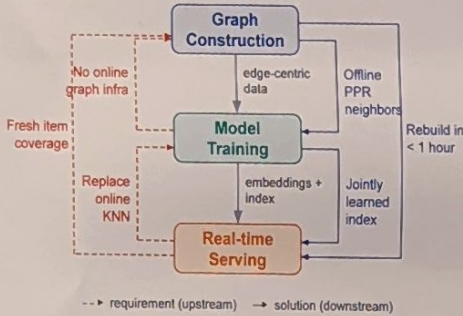
+0.96% / +2.75%

CTR / CVR, 14-day online A/B test ($p < 0.05$)

20+

production retrieval launches across major Meta surfaces

The idea: requirements cascade upstream, solutions flow downstream



Serving $\Rightarrow$ Training

U2U2I needs users active in the last 15 min; online KNN over that churn costs thousands of machines. $\rightarrow$ Training co-learns a cluster index, and each cluster holds a live item queue — so U2U2I is a lookup.

Training $\Rightarrow$ Construction

Graph stores and samplers are costly and make batch shapes unpredictable. $\rightarrow$ Neighborhoods move offline onto standard ML infrastructure, unlocking multi-hop PPR and popularity correction.

Serving $\Rightarrow$ Construction

Structural similarity is stable; fresh item coverage is not. $\rightarrow$ Full rebuild in ~ 1 h, refreshed every 3 h; ~ 2 h from engagement log to served embedding.

System	Stages optimized	Cross-stage gaps / couplings
GIGL, LIGNN	Training only	Graph assumed given; need online graph infrastructure; serve via ANN
MacGNN	Training only	Macro-node clustering loses micro-node precision; serves via ANN
RankGraph-2	All three, jointly	C $\rightarrow$ T: PPR neighbors, edge-centric data T $\rightarrow$ S: co-learned index S $\rightarrow$ C: 3 h refresh

1 Graph Construction

$\sim 10^{11} \rightarrow \sim 10^{11}$ edges, rebuilt in < 1 hour

Three edge types, engagement data only

- U-I (directed): user engaged the item within the past T hours; weight set by engagement type.
- U-U and I-I (undirected co-engagement): linked when the co-engagement count $\geq C_U / C_I$, with log-normalized weight

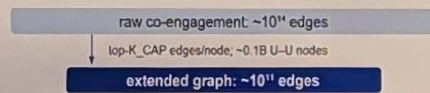
$$w(n_i, n_j) = \ln(\sum w(n_i, e) \cdot w(n_j, e))$$

- so heavy and light users stay on one scale.
- No external data — no social graph, no knowledge graph. One co-engagement relation per node-type pair beat many fine-grained ones.
- Subsampling preserves every node's exact topological position — unlike macro-node clustering, which folds tail users and niche items into generalised clusters.

Tractable, without erasing the tail

Popularity bias correction on I-I edges ($\alpha = 0.3$): if n_j is a hub, its edges are heavily down-weighted.

$$w(n_i, n_j) = w(n_i, n_j) \cdot (w(n_j, n_j) / \sum_k w(n_j, n_k))^{\alpha}$$



PPR pre-computation. Monte-Carlo walks (restart 0.15) pick the $K_{IMP} = 50$ most important neighbors per node; 10 are sampled per training edge. Output: self-contained edge records carrying both endpoints' features and neighbors.

Evidence

All three edge types are needed (user embeddings)

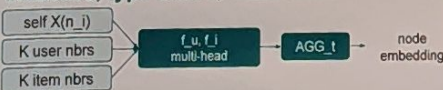


Popularity correction (item emb., Recall@100)
w/o correction 0.620
w/ correction 0.775
I-I edges lift long-tail items; U-U edges add discovery beyond a user's own history.

2 Model Training

A simple aggregator on a better graph — and zero graph infrastructure

Inductive, type-aware, multi-head



- Inductive — shared feature encoders generalize to nodes never seen in training, unlike transductive matrix factorization.
- Multi-head embeddings double as extra negatives; averaged at inference.
- Every node carries exactly K user and K item neighbors, fixed at graph-construction time — which is what makes batch shapes static.

Contrastive link prediction

- 100 negatives per positive edge: in-batch, an out-of-batch rolling pool approximating the global distribution, plus cross-head augmentation.
- Per-edge loss = λ margin $(0.1) + (1-\lambda) \cdot \text{InfoNCE}(\tau = 0.06)$, summed over four per-type losses; λ and the mixing weights β are learned by uncertainty weighting, never hand-tuned.

The payoff. No graph store, no sampler. Batch shapes are deterministic, keeping Model FLOPs Utilization high where online multi-hop sampling causes memory spikes.

Evidence

Neighbor selection dominates (user embeddings)



Random sampling is noisy; top-weight sees one hop only. PPR reaches structurally important nodes several hops out, at a cost paid once offline.

3 Co-Learned Index & Serving

U2U2I becomes U2Cluster2I — a lookup, not a search

Residual quantization, co-trained

Two codebook layers, $5,000 \times 50 = 250,000$ clusters. Each layer takes the nearest code and passes on the residual:

$$k_j = \argmin_j \|h_{(i-1)} - C_{(i,j)}\|^2, \quad h_i = h_{(i-1)} - C_{(i,k_i)} \\ h' = \sum_j C_{(i,k_j)}$$

The index is trained with the model, not bolted on after: alongside the reconstruction loss $\|h - h'\|^2$, the full contrastive objective is re-evaluated on h' , so the quantized embedding must itself be good for retrieval. All terms are balanced by uncertainty weighting.

Beating codebook collapse

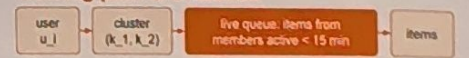
Under continuous training a few codes take over and the rest die. Two fixes:

- Balance regularizer. Track $\hat{p}$, code usage over the last 1,000 batches; penalize reinforcing already-popular codes via $L_{reg} = \hat{p} \cdot p_{batch}$.
- Biased code selection. Choose the relatively under-used code, not simply the nearest:

$$k_j = \argmax_j p(h_{(i-1)}, C_{(i,j)}) / \hat{p}_{(i,j)}$$

Result: 100% codebook utilization. The dispersion also aids discovery.

Serving path & evidence



Quantization costs almost nothing — but only with the regularizer:

	HR@1	HR@5	HR@10
Original embedding	0.871	0.981	0.997
Reconstructed (ours)	0.846	0.973	0.994
Reconstructed, no regularizer	0.717	0.877	0.931

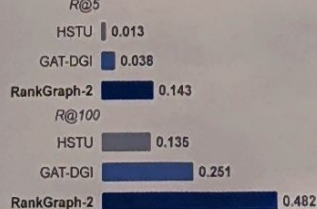
Online: the learned index matches approximate-KNN serving on CTR and CVR while cutting infrastructure by 83%.

Production results

Two major Meta surfaces, each serving billions of users

User embeddings

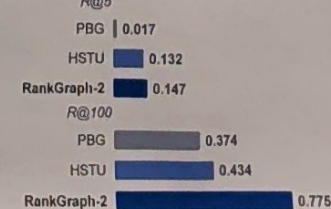
100K users, top-100 KNN vs. next-day engagement



3.8× GAT-DGI

Item embeddings

1,000 edges, day N vs. day N+1 co-engagement



2.1× PBG

Online A/B — 14 days, all results $p < 0.05$

Added as one more retrieval source alongside the existing production stack (collaborative filtering, content-based, other embedding retrieval).

U2I2I	CTR	CVR	U2U2I	CTR	CVR
Surface 1	+0.96%	+2.75%	Surface 1	+0.40%	+1.01%
Surface 2	+0.33%	+0.76%	Surface 2	+0.20%	+0.17%

System & training setup

- Graph from 24 h engagement, full rebuild every 3 h (~ 1 h each); ~ 2 h to a served embedding.
- Continuous training, batch 32,768; AdaGrad 0.02 sparse / AdamW 0.004 dense; dim 256.
- Billions of user and item nodes; hundreds of billions of edges.
- Index $5,000 \times 50$; $K_{IMP} = 50$; $K_{IMP} = 10$.